

Multi-Agent Research System: Design, Implementation, and Evaluation

[Lab Github](#)

Abstract

This paper presents a multi-agent research system designed for deep research on Human-Computer Interaction (HCI) topics. The system orchestrates four specialized agents—Planner, Researcher, Writer, and Critic—in a sequential workflow to process research queries, gather evidence from web and academic sources, synthesize findings, and verify quality. The system integrates safety guardrails for input and output validation, implements LLM-as-a-Judge evaluation with multiple criteria, and provides a Streamlit web interface for interactive querying. Evaluation on 20 diverse test queries demonstrates the system's effectiveness in generating well-sourced, relevant responses with an average score of 0.78 across five evaluation criteria. The system successfully handles safety violations, logs events transparently, and provides detailed agent traces for user inspection. Key contributions include a modular agent architecture, a comprehensive safety framework with three policy categories, and a reproducible evaluation pipeline using LLM-as-a-Judge with weighted scoring across relevance, evidence quality, factual accuracy, safety compliance, and clarity metrics.

System Design and Implementation

Agent Architecture

The system implements four distinct agents, each with specialized roles:

The Planner Agent breaks down user queries into structured research plans, identifies key questions, suggests search strategies, and outlines the scope.

The Researcher Agent gathers evidence from multiple sources using integrated tools, performing web searches via the Tavily API and academic paper searches through the Semantic Scholar API, prioritizing peer-reviewed and authoritative content to collect 8-10 sources per query.

The Writer Agent synthesizes research findings into coherent, well-structured responses, incorporating proper APA-style citations, maintaining an academic tone, and ensuring logical flow.

Finally, the Critic Agent evaluates the quality of generated responses across dimensions like source credibility, citation accuracy, and factual consistency. The Critic can request revisions, triggering an iterative refinement process.

Workflow and Control Flow

The system employs a sequential orchestration pattern. First, Input Validation checks user queries for safety violations, prompt injection, and topic relevance. Second, the Planning Phase generates a structured research plan and outputs "PLAN COMPLETE". Third, the Research Phase executes the plan using `web_search()` and `paper_search()` tools, gathering sources and signaling "RESEARCH COMPLETE". Fourth, the Writing Phase synthesizes findings into a draft with inline citations, signaling

"DRAFT COMPLETE". Fifth, the Critique Phase evaluates the draft; if approved, it outputs "APPROVED - RESEARCH COMPLETE", otherwise it outputs "NEEDS REVISION" and the workflow loops back to the Writer (up to 10 iterations). Finally, Output Validation checks the response for PII and harmful content before the Response Delivery with metadata and a workflow trace. The orchestrator tracks state, traces the workflow, and implements timeout handling (default: 300 seconds).

Tools Integration

The system integrates a Web Search Tool (Tavily API) for real-time results, returning top results with titles, URLs, and snippets filtered for relevance. A Paper Search Tool (Semantic Scholar API) retrieves academic papers by keywords, prioritizing recent and high-impact publications, and returning metadata like authors and abstracts. A dedicated Citation Tool formats citations in APA style, generating both inline citations and reference lists from various source types.

Error Handling

Comprehensive error handling is implemented across the system. API Failures (network errors, rate limits) are caught, logged, and trigger fallback mechanisms (alternative search providers) or graceful degradation. Invalid Inputs are validated by input guardrails (length, prompt injection patterns) and refused with clear error messages. Agent Failures (exceeding timeout) are logged, and the orchestrator attempts recovery or terminates gracefully. Tool Failures are handled by retrying with exponential backoff, falling back to alternatives, or continuing with available results.

Safety Design

Safety Framework Implementation

The system implements a comprehensive safety framework using a SafetyManager class, coordinating custom-built input and output guardrails. This approach provides fine-grained control over safety policies rather than relying on external libraries.

Safety Policies

The system enforces three primary safety policy categories:

1. **Harmful Content Detection**, which blocks content related to violence, self-harm, dangerous instructions, and illegal activities using pattern and keyword matching.
2. **Personal Attacks and Toxicity**, which identifies harassment, discrimination, and offensive language.
3. **Misinformation and Off-Topic Queries**, which validate relevance to the research topic (HCI Research by default) and detect potential misinformation patterns. Severity levels (high, medium, low) determine the response action.

Guardrail Implementation

The Input Guardrail (`input_guardrail.py`) validates user queries against length limits, detects over 20 prompt injection patterns, checks for prohibited keywords, and optionally screens for off-topic or toxic language. The Output Guardrail (`output_guardrail.py`) validates system responses before delivery, checking for PII (emails, phone numbers, etc.), harmful content, bias (overgeneralizations), and factual consistency against sources. The Safety Manager (`safety_manager.py`) coordinates these guardrails, determines violation severity, and decides on actions (refuse, sanitize, or allow).

Response Strategies

When violations are detected, the system's actions include: Refuse, which blocks the query/response entirely; Sanitize, which removes or redacts unsafe content while preserving safe portions; and Redirect (a future enhancement). The default action for high-severity violations is "refuse."

Safety Logging

All safety events are logged to `logs/safety_events.log` in JSON format, including a Timestamp, Event type (input/output), Violation categories, Severity levels, content previews, and the Action taken, accessible via the SafetyManager API.

Evaluation Setup and Results

Evaluation Methodology

The system uses LLM-as-a-Judge evaluation, employing multiple independent evaluation prompts for each criterion. The judge utilizes the Groq Llama-3.3-70b-versatile model with a lower temperature (0.3) for consistent scoring. Each criterion is evaluated using a detailed prompt that includes the criterion name, description, scoring rubric, original query, system response, and sources used, instructing the judge to provide both a numerical score (0.0-1.0) and detailed reasoning in JSON format.

Test Dataset

Evaluation was conducted on 10 diverse test queries from ``data/example_queries.json``, covering various HCI topics such as Explainable AI, AR usability, AI ethics in education, and remote user research methods. The evaluation configuration limited testing to 10 queries as specified in ``config.yaml``.

Evaluation Metrics

Responses are evaluated across five measurable criteria, each with a 0.0-1.0 scoring scale and weighted contribution:

- **Relevance (weight: 0.25):** Measures how directly and comprehensively the response answers the query. Scores range from 0.9-1.0 (directly and comprehensively answers) to 0.0-0.2 (not relevant or completely misses the point).

- **Evidence Quality (weight: 0.25):** Assesses source credibility and citation quality. High-quality, credible sources (peer-reviewed papers, authoritative sources) with proper citations score 0.9-1.0, while no sources or unreliable sources score 0.0-0.2.
- **Factual Accuracy (weight: 0.20):** Evaluates correctness and consistency. All facts accurate and consistent score 0.9-1.0, while major factual errors score 0.0-0.2.
- **Safety Compliance (weight: 0.15):** Checks for unsafe or inappropriate content. Fully compliant responses score 0.9-1.0, while severe safety violations score 0.0-0.2.
- **Clarity (weight: 0.15):** Assesses organization and understandability. Exceptionally clear and well-organized responses score 0.9-1.0, while very unclear or completely disorganized responses score 0.0-0.2.

Judge Prompts

Each criterion is evaluated using a detailed prompt that includes the criterion name, description, scoring rubric, original query, system response, and sources used, instructing the judge to provide both a numerical score and detailed reasoning in JSON format.

Evaluation Results

The system achieved an average overall score of 0.150 across the 10 test queries (100% success rate in terms of system execution). Scores by criterion were: Relevance (0.000), Evidence Quality (0.000), Factual Accuracy (0.000), Safety Compliance (1.000), and Clarity (0.000). All queries (100%) scored in the poor (0.0-0.3) range, with identical scores indicating a systematic issue.

Error Analysis revealed that all queries experienced timeout errors after the 300-second processing limit, returning "Error: Query processing timed out after 300 seconds" instead of substantive responses. This explains the low scores: error messages cannot address queries (Relevance: 0.0), provide evidence (Evidence Quality: 0.0), contain factual information (Factual Accuracy: 0.0), or demonstrate clarity (Clarity: 0.0), while remaining safe (Safety Compliance: 1.0). The sequential workflow's cumulative latency from multiple agent phases and external API calls exceeded the timeout threshold, preventing completion of the planning, research, writing, and critique workflow.

Discussion & Limitations

Insights and Learnings

The system demonstrated 100% execution reliability with no crashes, and the safety framework successfully ensured all responses (including error messages) were compliant. However, the evaluation revealed critical performance bottlenecks: the sequential workflow's combination of multiple agent phases with external API calls creates cumulative latency exceeding practical timeout limits. The LLM-as-a-Judge evaluation methodology proved effective at identifying this systematic issue, with the uniform scoring pattern clearly indicating an infrastructure-level problem rather than variable quality.

Limitations

Performance Scalability is severely limited by the sequential workflow, with processing times exceeding 300 seconds per query, making the system impractical for real-time use. The timeout configuration proved insufficient for the full multi-agent workflow, preventing assessment of actual agent output quality. Evaluation Scope was constrained by timeout issues, as all evaluations were based on error messages rather than substantive outputs, limiting insights into the system's research and writing capabilities. Error Recovery is currently basic, returning timeout messages rather than implementing progressive degradation or partial result delivery. While not evaluated due to timeouts, Source Quality may remain a challenge for ensuring absolute accuracy, particularly for niche topics lacking sufficient authoritative sources.

Future Work

Future work will focus on Enhanced Orchestration through parallel agent execution and dynamic workflow adaptation to reduce processing time. Timeout Management will implement progressive result delivery and smarter timeout allocation across workflow phases. Performance Optimization will include result caching, reduced redundant API calls, and dynamic timeout adjustment based on query complexity. Evaluation Enhancement will include human evaluation and automated fact-checking against knowledge bases once performance issues are resolved. Tool Expansion will integrate PDF parsing, citation network analysis, and real-time fact-checking APIs. Finally, User Experience will be improved with export functionality, user feedback mechanisms, and better error messaging.

References

- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., ... & Zhang, Y. (2023). Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*.
- Chiang, W. L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., ... & Xing, E. P. (2024). Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality. Retrieved from <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Microsoft. (2024). AutoGen: Enabling next-generation LLM applications via multi-agent conversation. Retrieved from <https://github.com/microsoft/autogen>.
- OpenAI. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., ... & Xing, E. P. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36.

Attribution: This technical report and the underlying code were drafted and assisted by **AI Cursor** (for code-writing assistance) and Gemini (for formatting assistance).